

Machine Learning Programs

1. For a given set of training data examples stored in a .CSV file, implement and demonstrate the Candidate-Elimination algorithm to output a description of the set of all hypotheses Consistent with the training examples.

```
import numpy as np
import pandas as pd

data = pd.DataFrame(data=pd.read_csv('/home/user/Documents/trainingdata.csv'))
print(data)
concepts = np.array(data.iloc[:,0:-1])
print(concepts)

target = np.array(data.iloc[:,-1])
print(target)
def learn(concepts, target):

    specific_h = concepts[0].copy()
    print("\nInitialization of specific_h and general_h")
    print(specific_h)

    general_h = [["?" for i in range(len(specific_h))] for i in range(len(specific_h))]
    print(general_h)
    for i, h in enumerate(concepts):
        if target[i] == "Yes":
            for x in range(len(specific_h)):
                if h[x] != specific_h[x]:
                    specific_h[x] = '?'
                    general_h[x][x] = '?'

        if target[i] == "No":
            for x in range(len(specific_h)):
                if h[x] != specific_h[x]:
                    general_h[x][x] = specific_h[x]
                else:
                    general_h[x][x] = '?'

    print("\nSteps of Candidate Elimination Algorithm",i+1)
    print(specific_h)
    print(general_h)

indices = [i for i, val in enumerate(general_h) if val == ['?', '?', '?', '?', '?', '?']]
for i in indices:
```

```

general_h.remove(['?', '?', '?', '?', '?', '?'])

return specific_h, general_h
s_final, g_final = learn(concepts, target)
print("\nFinal Specific_h:", s_final, sep="\n")
print("\nFinal General_h:", g_final, sep="\n")

```

OUTPUT:

Initialization of specific_h and general_h

```

['Sunny' 'Warm' 'Normal' 'Strong' 'Warm' 'Same']
[['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?']]

```

Steps of Candidate Elimination Algorithm 1

```

['Sunny' 'Warm' 'Normal' 'Strong' 'Warm' 'Same']
[['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?']]

```

Steps of Candidate Elimination Algorithm 2

```

['Sunny' 'Warm' '?' 'Strong' 'Warm' 'Same']
[['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?']]

```

Steps of Candidate Elimination Algorithm 3

```

['Sunny' 'Warm' '?' 'Strong' 'Warm' 'Same']
[['Sunny', '?', '?', '?', '?', '?'], ['?', 'Warm', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?']]

```

Steps of Candidate Elimination Algorithm 4

```

['Sunny' 'Warm' '?' 'Strong' '?' '?']
[['Sunny', '?', '?', '?', '?', '?'], ['?', 'Warm', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?']]

```

Final Specific_h:

```

['Sunny' 'Warm' '?' 'Strong' '?' '?']

```

Final General_h:

```

[['Sunny', '?', '?', '?', '?', '?'], ['?', 'Warm', '?', '?', '?', '?']]

```

2. Implement A* Search algorithm.

```
import heapq
GRID_SIZE = 10
grid = [[0 for _ in range(GRID_SIZE)] for _ in range(GRID_SIZE)]
start = (0, 0)
goal = (GRID_SIZE - 1, GRID_SIZE - 1)
def heuristic(node, goal):
    return abs(node[0] - goal[0]) + abs(node[1] - goal[1])

def astar(grid, start, goal):
    open_list = []
    closed_list = set()

    cost = {start: 0}
    prev_node = {start: None}
    heapq.heappush(open_list, (0, start))

    while open_list:
        current_cost, current_node = heapq.heappop(open_list)

        if current_node == goal:
            path = []
            while current_node is not None:
                path.append(current_node)
                current_node = prev_node[current_node]
            return path[::-1]
        closed_list.add(current_node)
        for dx, dy in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
            neighbor = (current_node[0] + dx, current_node[1] + dy)
            if 0 <= neighbor[0] < GRID_SIZE and 0 <= neighbor[1] < GRID_SIZE:
                if grid[neighbor[0]][neighbor[1]] == 0 and neighbor not in closed_list:
                    neighbor_cost = cost[current_node] + 1

                    if neighbor not in cost or neighbor_cost < cost[neighbor]:
                        cost[neighbor] = neighbor_cost
                        prev_node[neighbor] = current_node
                        heapq.heappush(open_list, (neighbor_cost + heuristic(neighbor, goal), neighbor))

    return None
path = astar(grid, start, goal)

if path is not None:
    print("Path found:", path)
```

```
else:  
    print("No path found")
```

OUTPUT:

Path found: [(0, 0), (0, 1), (0, 2), (0, 3), (0, 4), (0, 5), (0, 6), (0, 7), (0, 8), (0, 9), (1, 9), (2, 9), (3, 9), (4, 9), (5, 9), (6, 9), (7, 9), (8, 9), (9, 9)]

3.Implement A python program to implement Simple linear regression using Least Square Method

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Load the dataset
hours_studied = np.array([2, 4, 6, 8, 10, 12, 14, 16, 18, 20])
exam_score = np.array([40, 50, 60, 70, 80, 90, 100, 110, 120, 130])

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(hours_studied.reshape(-1, 1), exam_score,
test_size=0.2, random_state=42)

# Create a Simple Linear Regression model
model = LinearRegression()

# Train the model
model.fit(X_train, y_train)

# Make predictions
y_pred = model.predict(X_test)

# Evaluate the model's performance
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print(f'MSE: {mse:.2f}')
print(f'R2: {r2:.2f}')
```

OUTPUT:

MSE: 0.00
R2: 1.00

4.Implement a decision tree algorithm for sales prediction/ classification in retail sector.

```
import pandas as pd# Define the dataset
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([0, 1, 1, 0])

# Define the learning rate and number of iterations
learning_rate = 0.1
num_ iterations = 1000

# Initialize the weights and bias randomly
weights = np.random.rand(2)
bias = np.random.rand(1)
# Train the perceptron
for i in range(num_ iterations):
    # Calculate the output of the perceptron
    output = np.dot(X, weights) + bias

    # Apply the activation function (threshold function)
    output = np.where(output > 0, 1, 0)

    # Calculate the error
    error = y - output

    # Update the weights and bias
    weights += learning_rate * np.dot(X.T, error)
    bias += learning_rate * np.sum(error)
# Test the perceptron
test_X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
test_output = np.dot(test_X, weights) + bias
test_output = np.where(test_output > 0, 1, 0)
print("Test Output:", test_output)
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report

data = {
    "Customer_Segment": ["Gold", "Silver", "Bronze", "Gold", "Silver", "Bronze"],
    "Product_Category": ["Electronics", "Clothing", "Food", "Home Decor", "Clothing",
"Electronics"],
    "Purchase_Amount": [50, 15, 80, 30, 20, 10],
    "Promotion_Used": ["Yes", "No", "No", "Yes", "Yes", "No"],
    "Sales_Target": ["High", "Medium", "Low", "Medium", "Medium", "Low"]
}
df = pd.DataFrame(data)
```

```

df["Customer_Segment"] = pd.Categorical(df["Customer_Segment"]).codes
df["Product_Category"] = pd.Categorical(df["Product_Category"]).codes
df["Promotion_Used"] = pd.Categorical(df["Promotion_Used"]).codes

# Separate features and target variable
X = df[["Customer_Segment", "Product_Category", "Purchase_Amount", "Promotion_Used"]]
y = df["Sales_Target"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = DecisionTreeClassifier()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))

```

OUTPUT:

```

Accuracy: 0.5
      precision  recall  f1-score  support

   High      0.00    0.00    0.00         1
  Medium      0.50    1.00    0.67         1

 accuracy                0.50         2
 macro avg      0.25    0.50    0.33         2
 weighted avg      0.25    0.50    0.33         2

```

5. Implement back propagation algorithm for stock prices prediction

```
import numpy as np
X = np.array([[2, 9], [1, 5], [3, 6]], dtype=float)
y = np.array([[92], [86], [89]], dtype=float)
X = X / np.amax(X, axis=0)
y = y / 100
# Sigmoid Function
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def derivatives_sigmoid(x):
    return x * (1 - x)

epoch = 5000
lr = 0.1
inputlayer_neurons = 2
hiddenlayer_neurons = 3
output_neurons = 1
wh = np.random.uniform(size=(inputlayer_neurons, hiddenlayer_neurons))
bh = np.random.uniform(size=(1, hiddenlayer_neurons))
wout = np.random.uniform(size=(hiddenlayer_neurons, output_neurons))
bout = np.random.uniform(size=(1, output_neurons))

for i in range(epoch):

    hinp1 = np.dot(X, wh)
    hinp = hinp1 + bh
    hlayer_act = sigmoid(hinp)
    outinp1 = np.dot(hlayer_act, wout)
    outinp = outinp1 + bout
    output = sigmoid(outinp)

    EO = y - output
    outgrad = derivatives_sigmoid(output)
    d_output = EO * outgrad
    EH = d_output.dot(wout.T)

    hiddengrad = derivatives_sigmoid(hlayer_act)
    d_hiddenlayer = EH * hiddengrad
    wout += hlayer_act.T.dot(d_output) * lr
    wh += X.T.dot(d_hiddenlayer) * lr

    print("Input: \n" + str(X))
    print("Actual Output: \n" + str(y))
    print("Predicted Output: \n", output)
```


OUTPUT:

Input:

```
[[0.66666667 1.      ]  
 [0.33333333 0.55555556]  
 [1.      0.66666667]]
```

Input:

```
[[0.66666667 1.      ]  
 [0.33333333 0.55555556]  
 [1.      0.66666667]]
```

Input:

```
[[0.66666667 1.      ]  
 [0.33333333 0.55555556]  
 [1.      0.66666667]]
```

Input:

```
[[0.66666667 1.      ]  
 [0.33333333 0.55555556]  
 [1.      0.66666667]]
```

Input:

```
[[0.66666667 1.      ]  
 [0.33333333 0.55555556]  
 [1.      0.66666667]]
```

Input:

```
[[0.66666667 1.      ]  
 [0.33333333 0.55555556]  
 [1.      0.66666667]]
```

Input:

```
[[0.66666667 1.      ]  
 [0.33333333 0.55555556]  
 [1.      0.66666667]]
```

Input:

```
[[0.66666667 1.      ]  
 [0.33333333 0.55555556]  
 [1.      0.66666667]]
```

Input:

```
[[0.66666667 1.      ]  
 [0.33333333 0.55555556]  
 [1.      0.66666667]]
```

Input:

```
[[0.66666667 1.      ]  
 [0.33333333 0.55555556]  
 [1.      0.66666667]]
```

Input:

```
[[0.66666667 1.      ]  
 [0.33333333 0.55555556]  
 [1.      0.66666667]]
```

6. Implement A python program to implement single layer perceptron.

```
import numpy as np
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([0, 1, 1, 0])
learning_rate = 0.1
num_iterations = 1000
weights = np.random.rand(2)
bias = np.random.rand(1)
for i in range(num_iterations):
    output = np.dot(X, weights) + bias
    output = np.where(output > 0, 1, 0)
    error = y - output
    weights += learning_rate * np.dot(X.T, error)
    bias += learning_rate * np.sum(error)
test_X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
test_output = np.dot(test_X, weights) + bias
test_output = np.where(test_output > 0, 1, 0)
print("Test Output:", test_output)
```

OUTPUT:

Test Output: [1 1 1 1]

7. Write a program to construct a Bayesian network considering medical data. Use this model to demonstrate the diagnosis of heart patients using standard Heart Disease Data Set. You can use Java/Python ML library classes/API.

```
import pandas as pd
data=pd.read_csv("/home/user/Documents/heartdisease.csv")
heart_disease=pd.DataFrame(data)
print(heart_disease)
from pgmpy.models import BayesianModel
model=BayesianModel([
('age','Lifestyle'),
('Gender','Lifestyle'),
('Family','heartdisease'),
('diet','cholesterol'),
('Lifestyle','diet'),
('cholesterol','heartdisease'),
('diet','cholesterol')])
from pgmpy.estimators import MaximumLikelihoodEstimator
model.fit(heart_disease, estimator=MaximumLikelihoodEstimator)

from pgmpy.inference import VariableElimination
HeartDisease_infer = VariableElimination(model)
print('For age Enter { SuperSeniorCitizen:0, SeniorCitizen:1, MiddleAged:2, Youth:3, Teen:4 }')
print('For Gender Enter { Male:0, Female:1 }')
print('For Family History Enter { yes:1, No:0 }')
print('For diet Enter { High:0, Medium:1 }')
print('For lifeStyle Enter { Athlete:0, Active:1, Moderate:2, Sedentary:3 }')
print('For cholesterol Enter { High:0, BorderLine:1, Normal:2 }')

q = HeartDisease_infer.query(variables=['heartdisease'], evidence={
    'age':int(input('Enter age :')),
    'Gender':int(input('Enter Gender :')),
    'Family':int(input('Enter Family history :')),
    'diet':int(input('Enter diet :')),
    'Lifestyle':int(input('Enter Lifestyle :')),
    'cholesterol':int(input('Enter cholesterol :'))
})

print(q['heartdisease'])
```

OUTPUT:

	age	Gender	Family	diet	Lifestyle	cholesterol	heartdisease
0	0	0	1	1	3	0	1
1	0	1	1	1	3	0	1
2	1	0	0	0	2	1	1
3	4	0	1	1	3	2	0
4	3	1	1	0	0	2	0
5	2	0	1	1	1	0	1
6	4	0	1	0	2	0	1
7	0	0	1	1	3	0	1
8	3	1	1	0	0	2	0
9	1	1	0	0	0	2	1
10	4	1	0	1	2	0	1
11	4	0	1	1	3	2	0
12	2	1	0	0	0	0	0
13	2	0	1	1	1	0	1
14	3	1	1	0	0	1	0
15	0	0	1	0	0	2	1
16	1	1	0	1	2	1	1
17	3	1	1	1	0	1	0
18	4	0	1	1	3	2	0

/home/user/anaconda3/lib/python3.9/site-packages/pgmpy/models/BayesianModel.py:8: FutureWarning: BayesianModel has been renamed to BayesianNetwork. Please use BayesianNetwork class, BayesianModel will be removed in future.

```
warnings.warn(
For age Enter { SuperSeniorCitizen:0, SeniorCitizen:1, MiddleAged:2, Youth:3, Teen:4 }
For Gender Enter { Male:0, Female:1 }
For Family History Enter { yes:1, No:0 }
For diet Enter { High:0, Medium:1 }
For lifeStyle Enter { Athlete:0, Active:1, Moderate:2, Sedentary:3 }
For cholesterol Enter { High:0, BorderLine:1, Normal:2 }
Enter age :
```

- 8. a) Implement Random Forest ensemble method on a given dataset.**
b) Implement Boosting ensemble method on a given dataset

a) Implement Random Forest ensemble method on a given dataset.

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_iris
from sklearn.metrics import accuracy_score, classification_report

# Load the Iris dataset
iris = load_iris()
X = iris.data
y = iris.target

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# Create a Random Forest classifier
rf = RandomForestClassifier(n_estimators=100, random_state=42)

# Train the classifier
rf.fit(X_train, y_train)

# Make predictions on the testing set
y_pred = rf.predict(X_test)

# Evaluate the classifier
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
print("Classification Report:")
print(classification_report(y_test, y_pred))
```

OUTPUT:

Accuracy: 1.0

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	1.00	1.00	9
2	1.00	1.00	1.00	11
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30

b) Implement Boosting ensemble method on a given dataset.

```
from sklearn.ensemble import AdaBoostClassifier
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_iris
from sklearn.metrics import accuracy_score, classification_report

# Load the Iris dataset
iris = load_iris()
X = iris.data
y = iris.target

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Create an AdaBoost classifier
abc = AdaBoostClassifier(n_estimators=100, random_state=42)
# Train the classifier
abc.fit(X_train, y_train)
# Make predictions on the testing set
y_pred = abc.predict(X_test)

# Evaluate the classifier
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
print("Classification Report:")
print(classification_report(y_test, y_pred))
```

OUTPUT:

Accuracy: 0.9333333333333333

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	0.89	0.89	0.89	9
2	0.91	0.91	0.91	11
accuracy			0.93	30
macro avg	0.93	0.93	0.93	30
weighted avg	0.93	0.93	0.93	30

9. Apply EM algorithm to cluster a set of data stored in a .CSV file. Use the same dataset for clustering using k Means algorithm. Compare the results of these two algorithms and comment on the quality of clustering. You can add Java/Python ML library classes/API in the program.

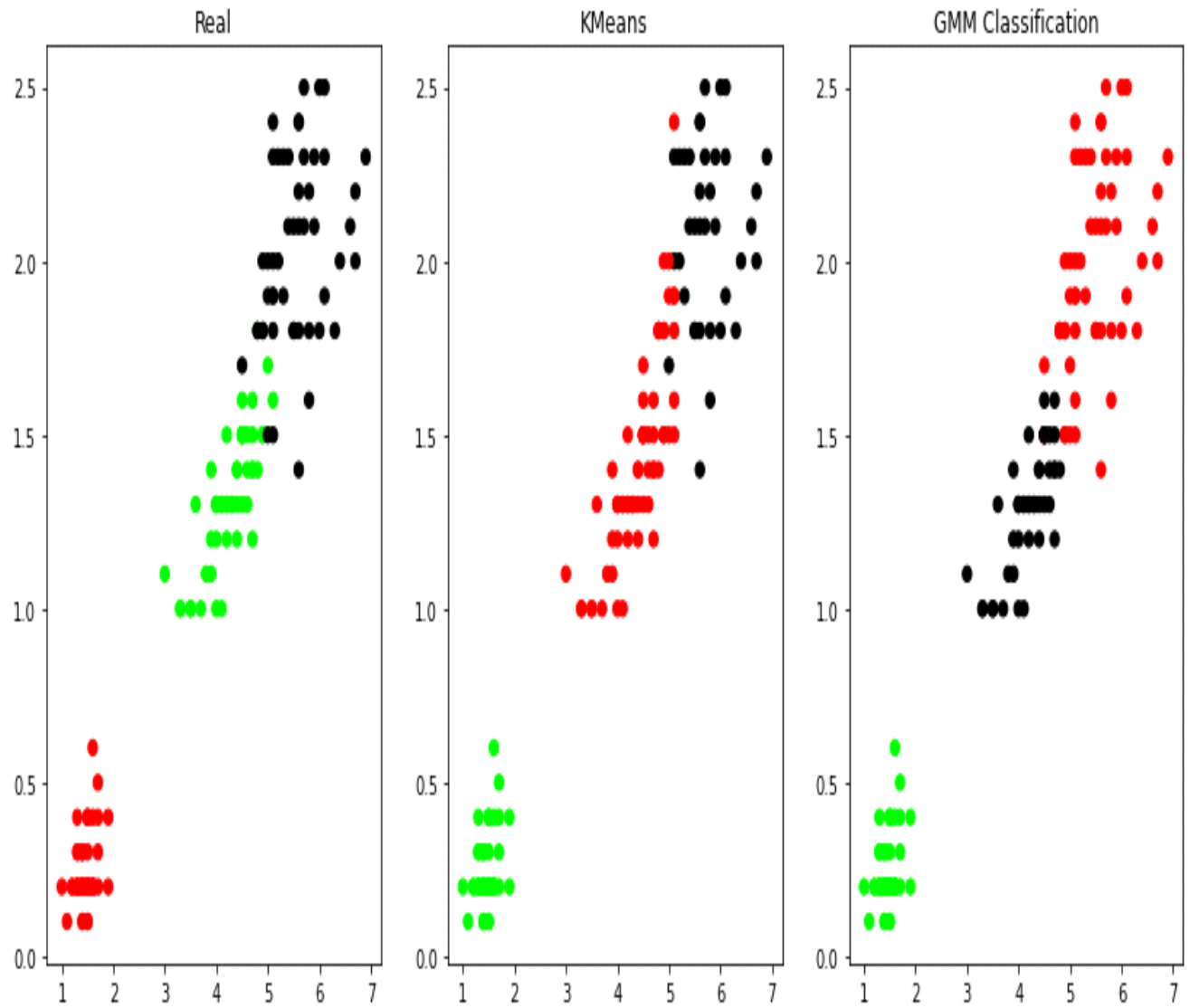
```
from sklearn.cluster import KMeans
from sklearn import preprocessing
from sklearn.mixture import GaussianMixture
from sklearn.datasets import load_iris
import sklearn.metrics as sm
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
dataset=load_iris()
# print(dataset)
X=pd.DataFrame(dataset.data)
X.columns=['Sepal_Length','Sepal_Width','Petal_Length','Petal_Width']
y=pd.DataFrame(dataset.target)
y.columns=['Targets']
# print(X)
plt.figure(figsize=(14,7))
colormap=np.array(['red','lime','black'])

# REAL PLOT
plt.subplot(1,3,1)
plt.scatter(X.Petal_Length,X.Petal_Width,c=colormap[y.Targets],s=40)
plt.title('Real')

# K-PLOT
plt.subplot(1,3,2)
model=KMeans(n_clusters=3)
model.fit(X)
predY=np.choose(model.labels_,[0,1,2]).astype(np.int64)
plt.scatter(X.Petal_Length,X.Petal_Width,c=colormap[predY],s=40)
plt.title('KMeans')
scaler=preprocessing.StandardScaler()
scaler.fit(X)
xsa=scaler.transform(X)
xs=pd.DataFrame(xsa,columns=X.columns)
gmm=GaussianMixture(n_components=3)
gmm.fit(xs)
y_cluster_gmm=gmm.predict(xs)
plt.subplot(1,3,3)
plt.scatter(X.Petal_Length,X.Petal_Width,c=colormap[y_cluster_gmm],s=40)
plt.title('GMM Classification')
```

OUTPUT:

Text(0.5, 1.0, 'GMM Classification')



10. Write a program to implement k-Nearest Neighbor algorithm to classify the iris data set. Print both correct and wrong predictions. Java/Python ML library classes can be used for this problem.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report

iris = load_iris()
X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

knn = KNeighborsClassifier(n_neighbors=5)

knn.fit(X_train, y_train)

y_pred = knn.predict(X_test)

print("Correct Predictions:")
for i in range(len(y_test)):
    if y_test[i] == y_pred[i]:
        print(f'Sample {i+1}: Actual={y_test[i]}, Predicted={y_pred[i]}')

print("\nWrong Predictions:")
for i in range(len(y_test)):
    if y_test[i] != y_pred[i]:
        print(f'Sample {i+1}: Actual={y_test[i]}, Predicted={y_pred[i]}')

accuracy = accuracy_score(y_test, y_pred)
print("\nAccuracy:", accuracy)
print("Classification Report:")
print(classification_report(y_test, y_pred))
```

OUTPUT:

Correct Predictions:

Sample 1: Actual=1, Predicted=1
Sample 2: Actual=0, Predicted=0
Sample 3: Actual=2, Predicted=2
Sample 4: Actual=1, Predicted=1
Sample 5: Actual=1, Predicted=1
Sample 6: Actual=0, Predicted=0
Sample 7: Actual=1, Predicted=1
Sample 8: Actual=2, Predicted=2
Sample 9: Actual=1, Predicted=1
Sample 10: Actual=1, Predicted=1
Sample 11: Actual=2, Predicted=2
Sample 12: Actual=0, Predicted=0
Sample 13: Actual=0, Predicted=0
Sample 14: Actual=0, Predicted=0
Sample 15: Actual=0, Predicted=0
Sample 16: Actual=1, Predicted=1
Sample 17: Actual=2, Predicted=2
Sample 18: Actual=1, Predicted=1
Sample 19: Actual=1, Predicted=1
Sample 20: Actual=2, Predicted=2
Sample 21: Actual=0, Predicted=0
Sample 22: Actual=2, Predicted=2
Sample 23: Actual=0, Predicted=0
Sample 24: Actual=2, Predicted=2
Sample 25: Actual=2, Predicted=2
Sample 26: Actual=2, Predicted=2
Sample 27: Actual=2, Predicted=2
Sample 28: Actual=2, Predicted=2
Sample 29: Actual=0, Predicted=0
Sample 30: Actual=0, Predicted=0

Wrong Predictions:

Accuracy: 1.0

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	1.00	1.00	9
2	1.00	1.00	1.00	11
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30